

**LAXMI CHARITABLE TRUST'S
SHETH L.U.J COLLEGE OF ARTS & SIR M.V. COLLEGE OF SCIENCE AND
COMMERCE
DEPARTMENT OF INFORMATION TECHNOLOGY**

BUSINESS ANALYTICS PROJECT

**BUSINESS ANALYTICS FOR SUPERMARKET:
CUSTOMER AND SALES ANALYSIS**

Submitted By:

Name: Sagar Kandari

Roll No.: T010

Class: TYIT

Division / Batch: A / 2

Subject: Business Analytics

Academic Year: 2026–2027

Faculty Name: Chetna Tanvade

1. Introduction

Business Analytics is the use of data, statistics and software to understand business performance and support better decisions. This project analyzes the sales data of a supermarket chain using Python and applies data preprocessing, Exploratory Data Analysis, descriptive statistics, correlation, regression, probability, decision analysis, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series forecasting, optimization, simulation, ANOVA and factorial experiments.

Data shows what customers buy, when they buy and how much they spend, and decisions based on data are more accurate than decisions based on guesses. A supermarket produces a large number of daily transactions, so it gives enough data for every technique used in this project. Analytics can show the best-selling products, the strongest branches, customer spending patterns, future sales and the best stock level to keep.

2. Problem Statement and Objectives

The supermarket does not know which products, branches and customers bring the most sales. This project analyzes supermarket sales data to identify customer purchasing patterns, the factors that affect sales, future sales trends and suitable business strategies. The data available is 500 invoices (January–March 2026) with product, branch, price, quantity, payment type, rating and sales. The decisions needed are which products to promote, how much stock to keep, and which payment and marketing methods to focus on. The objectives of this project are:

- To collect and clean the supermarket sales data.
- To perform exploratory data analysis.
- To identify relationships among business variables.
- To predict future sales.
- To perform statistical testing.
- To analyze customer behaviour.
- To identify suitable business strategies.
- To support business decision-making using data.

3. Business Scenario

The business studied is **FreshMart Supermarket** (a hypothetical business), a retail supermarket chain.

- **Type:** Retail supermarket with 3 branches – Andheri, Borivali and Thane.
- **Products:** Groceries, Beverages, Snacks, Personal Care and Home Care.
- **Target customers:** Families and working professionals (Member and Normal customers).
- **Operations:** Daily billing and stock purchase, with payment through Cash, Card and UPI.
- **Important problems:** Choosing stock levels, finding high-value products and planning promotions.

4. Dataset and Tools

The main dataset is **supermarket_sales.csv** (500 records after cleaning, 11 columns). It is business-generated point-of-sale billing data prepared for academic use, with the same layout as public supermarket sales datasets, covering **1 January 2026 to 31 March 2026**. The notebook initially loads 503 rows, including 3 duplicates, which are removed during preprocessing. The dataset is **sample data created for this project**; it does not belong to a real company.

Column	Description	Data Type
Invoice_ID	Unique invoice number	Categorical
Date	Date of purchase	Date/Time
Branch	Store branch (Andheri, Borivali, Thane)	Categorical
Customer_Type	Member or Normal	Categorical
Gender	Customer gender	Categorical
Product_Line	Product category	Categorical
Unit_Price	Price of one unit (Rs.)	Numerical
Quantity	Units purchased	Numerical
Payment	Cash, Card or UPI	Categorical
Rating	Customer rating (4–10)	Numerical
Sales	Unit_Price × Quantity (Rs.)	Numerical

Numerical variables: Unit_Price, Quantity, Rating, Sales. **Categorical variables:** Invoice_ID, Branch, Customer_Type, Gender, Product_Line, Payment. **Date/time variable:** Date.

Item	Details
Language	Python
Environment	Jupyter Notebook
Libraries	Pandas, NumPy, Matplotlib, Seaborn, SciPy, Scikit-learn, Statsmodels
Data format	CSV
BI tools	Tableau / Power BI (dashboard_data.csv)

5. Setup, Data Loading and Preprocessing

5.1 Import Libraries and Load Dataset

The required libraries are imported. If the dataset file is not present, a sample supermarket dataset (with a few missing values and duplicate rows) is created, and then the file is loaded.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import os

# If the dataset file is not present, create a sample supermarket dataset
if not os.path.exists("supermarket_sales.csv"):
    np.random.seed(42)
    n = 500
    lines = ["Groceries", "Beverages", "Snacks", "Personal Care", "Home Care"]
    price_range = {"Groceries": (20, 120), "Beverages": (15, 90), "Snacks": (10, 60),
                  "Personal Care": (40, 250), "Home Care": (30, 200)}
    product = np.random.choice(lines, n)
```

```

price = [round(np.random.uniform(*price_range[p]), 2) for p in product]
data = pd.DataFrame({
    "Invoice_ID": ["INV" + str(1000 + i) for i in range(n)],
    "Date": np.random.choice(pd.date_range("2026-01-01", "2026-03-31").strftime("%Y-%m-%d"), n),
    "Branch": np.random.choice(["Andheri", "Borivali", "Thane"], n),
    "Customer_Type": np.random.choice(["Member", "Normal"], n, p=[0.45, 0.55]),
    "Gender": np.random.choice(["Male", "Female"], n),
    "Product_Line": product,
    "Unit_Price": price,
    "Quantity": np.random.randint(1, 11, n),
    "Payment": np.random.choice(["Cash", "Card", "UPI"], n, p=[0.25, 0.30, 0.45]),
    "Rating": np.round(np.random.uniform(4, 10, n), 1)}))
data["Sales"] = (data["Unit_Price"] * data["Quantity"]).round(2)
data.loc[[10, 55, 120, 300, 410], "Rating"] = np.nan           # a few missing values
data = pd.concat([data, data.iloc[[3, 77, 200]]])              # a few duplicate rows
data.to_csv("supermarket_sales.csv", index=False)

df = pd.read_csv("supermarket_sales.csv")
print("Dataset loaded successfully")
print("Records and Columns:", df.shape)
print("Sagar Kandari T010")

```

Output

```

Dataset loaded successfully
Records and Columns: (503, 11)
Sagar Kandari T010

```

5.2 Data Preprocessing

Missing values, duplicate records, data types, invalid values, outliers and inconsistent text values are checked and handled.

Code

```

# 1. Missing values
print("Missing Values:")
print(df.isnull().sum())

# 2. Duplicate records
print("\nDuplicate Records:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

# Fill missing Rating with median
df["Rating"] = df["Rating"].fillna(df["Rating"].median())

# 3. Correct data type of Date
df["Date"] = pd.to_datetime(df["Date"])

# 4. Invalid values (price or quantity zero or negative)
invalid = df[(df["Unit_Price"] <= 0) | (df["Quantity"] <= 0)]
print("Invalid Records:", len(invalid))

# 5. Outliers in Sales using IQR
Q1 = df["Sales"].quantile(0.25)
Q3 = df["Sales"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df["Sales"] < lower) | (df["Sales"] > upper)]
print("Outliers in Sales:", len(outliers))

# 6. Inconsistent text values

```

```

for col in ["Branch", "Customer_Type", "Gender", "Product_Line", "Payment"]:
    df[col] = df[col].str.strip().str.title()

print("\nAfter cleaning:")
print("Shape:", df.shape)
print(df.isnull().sum().sum(), "missing values left")
print("Sagar Kandari T010")

```

Output

```

Missing Values:
Invoice_ID      0
Date            0
Branch          0
Customer_Type   0
Gender          0
Product_Line    0
Unit_Price      0
Quantity        0
Payment         0
Rating          5
Sales           0
dtype: int64

Duplicate Records: 3
Invalid Records: 0
Outliers in Sales: 35

After cleaning:
Shape: (500, 11)
0 missing values left
Sagar Kandari T010

```

Observation: 5 missing Ratings were filled with the median, 3 duplicate rows were removed, the Date column was converted to date format, and no invalid values were found. 35 high-value invoices were identified as outliers; they are genuine bulk purchases, so they were kept. Text fields were stripped and standardized, leaving 0 missing values and 0 duplicate records.

6. Exploratory Data Analysis

6.1 Dataset Overview

The first and last five records, shape, column names, data types, missing values, duplicates, unique values and basic statistics are displayed.

Code

```

print("First Five Records:")
print(df.head())
print("\nLast Five Records:")
print(df.tail())
print("\nShape:", df.shape)
print("\nColumn Names:", list(df.columns))
print("\nData Types:")

```

```

print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())
print("\nUnique Values:")
print(df.nunique())
print("\nBasic Statistics:")
print(df.describe())
print("Sagar Kandari T010")

```

Output

```

First Five Records:
  Invoice_ID      Date      Branch  ... Payment Rating  Sales
0  INV1000  2026-01-19  Andheri    ...    Upi      5.9   584.40
1  INV1001  2026-02-05    Thane    ...    Upi      5.0   341.00
2  INV1002  2026-01-29  Borivali    ...    Upi      5.5   130.00
3  INV1003  2026-03-01  Borivali    ...   Cash      5.6   546.72
4  INV1004  2026-03-23  Andheri    ...   Card      6.0   769.28

[5 rows x 11 columns]

Last Five Records:
  Invoice_ID      Date      Branch  ... Payment Rating  Sales
495  INV1495  2026-01-17  Andheri    ...   Cash      5.0   328.89
496  INV1496  2026-03-25  Borivali    ...    Upi      5.4    95.85
497  INV1497  2026-01-09    Thane    ...    Upi      8.3    57.10
498  INV1498  2026-02-07    Thane    ...   Cash      5.6    81.58
499  INV1499  2026-02-17  Andheri    ...   Cash      6.0   158.55

[5 rows x 11 columns]

Shape: (500, 11)

Column Names: ['Invoice_ID', 'Date', 'Branch', 'Customer_Type', 'Gender', 'Product_Line',
               'Unit_Price', 'Quantity', 'Payment', 'Rating', 'Sales']

Data Types:
Invoice_ID      str
Date            datetime64[us]
Branch          str
Customer_Type   str
Gender          str
Product_Line    str

```

```
Unit_Price      float64
Quantity        int64
Payment         str
Rating          float64
Sales           float64
dtype: object
```

Missing Values:

```
Invoice_ID      0
Date            0
Branch          0
Customer_Type   0
Gender          0
Product_Line    0
Unit_Price      0
Quantity        0
Payment         0
Rating          0
Sales           0
dtype: int64
```

Duplicate Records: 0

Unique Values:

```
Invoice_ID      500
Date            90
Branch          3
Customer_Type    2
Gender          2
Product_Line     5
Unit_Price      489
Quantity        10
```

```
Payment         3
Rating          61
Sales           497
dtype: int64
```

Basic Statistics:

	Date	Unit_Price	...	Rating	Sales
count	500	500.00000	...	500.000000	500.00000
mean	2026-02-13 13:03:21.600000	85.76394	...	7.146400	455.41966
min	2026-01-01 00:00:00	10.57000	...	4.000000	13.05000
25%	2026-01-22 00:00:00	40.09250	...	5.800000	157.27000
50%	2026-02-12 12:00:00	66.16500	...	7.200000	311.06000
75%	2026-03-07 00:00:00	118.15000	...	8.500000	588.73750
max	2026-03-31 00:00:00	249.34000	...	10.000000	2283.30000
std	NaN	60.06724	...	1.676122	427.79348

```
[8 rows x 5 columns]
Sagar Kandari T010
```

6.2 Sales by Product Line and Branch

Two bar charts compare total sales across product lines and across branches.

Code

```
# Bar chart 1: Sales by Product Line
plt.figure(figsize=(7,5))
df.groupby("Product_Line")["Sales"].sum().sort_values().plot(kind="bar", color="steelblue")
plt.title("Total Sales by Product Line")
plt.xlabel("Product Line")
```

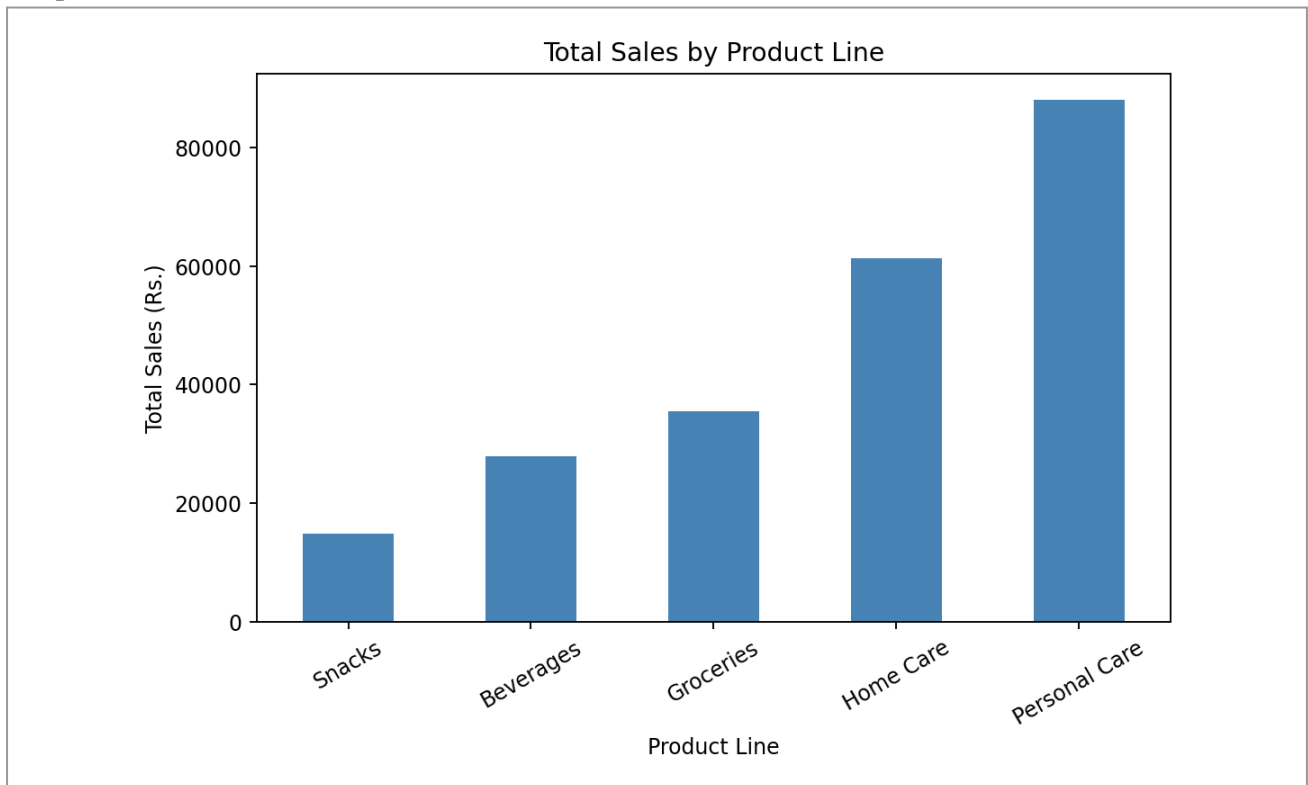
```

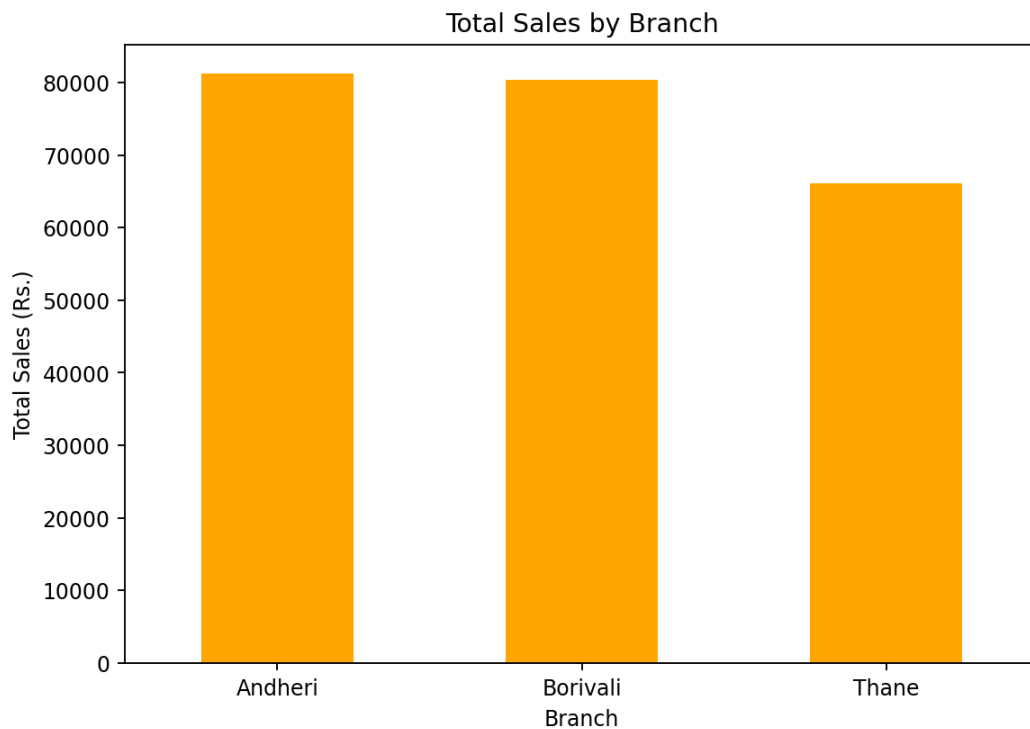
plt.ylabel("Total Sales (Rs.)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# Bar chart 2: Sales by Branch
plt.figure(figsize=(7,5))
df.groupby("Branch")["Sales"].sum().plot(kind="bar", color="orange")
plt.title("Total Sales by Branch")
plt.xlabel("Branch")
plt.ylabel("Total Sales (Rs.)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")

```

Output





Sagar Kandari T010

Observation: Personal Care and Home Care bring the highest total sales. Snacks bring the lowest. Andheri and Borivali have almost equal sales, while Thane is lower.

6.3 Sales and Rating Histograms

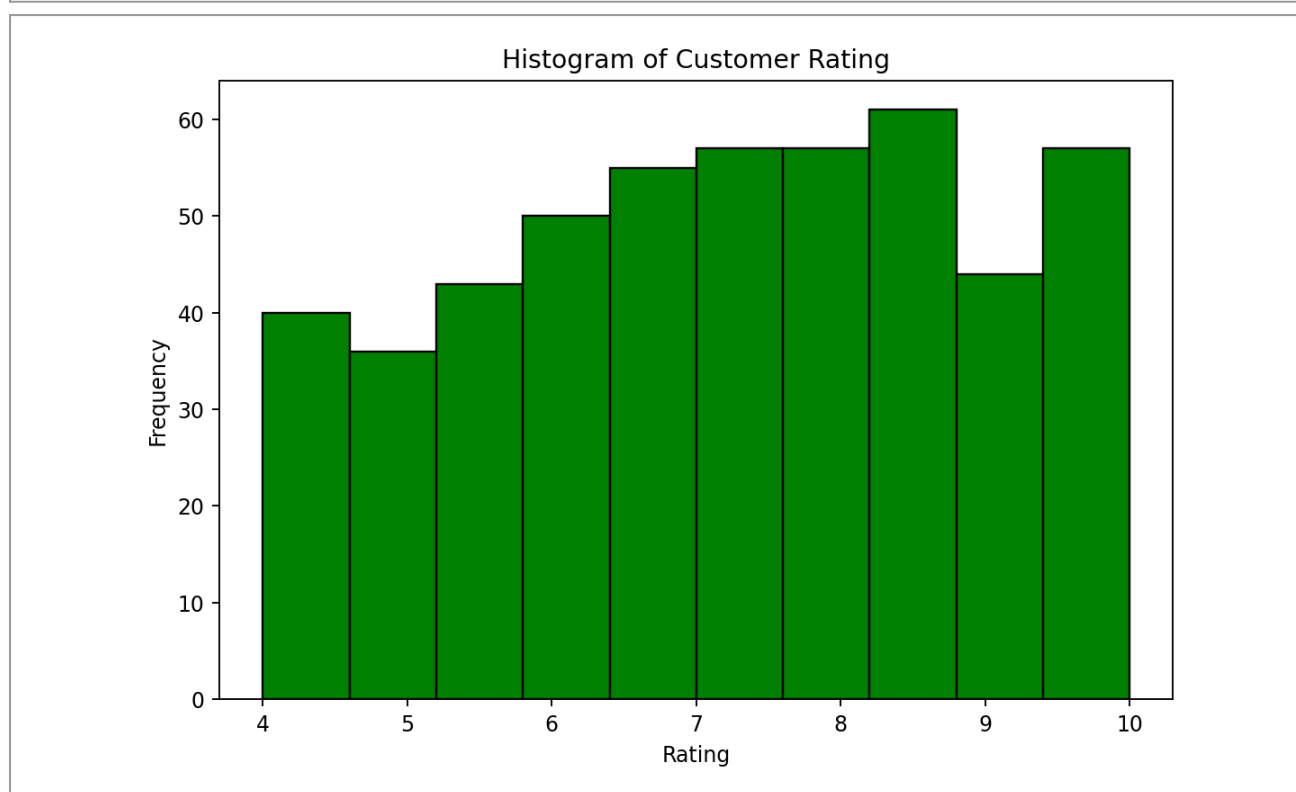
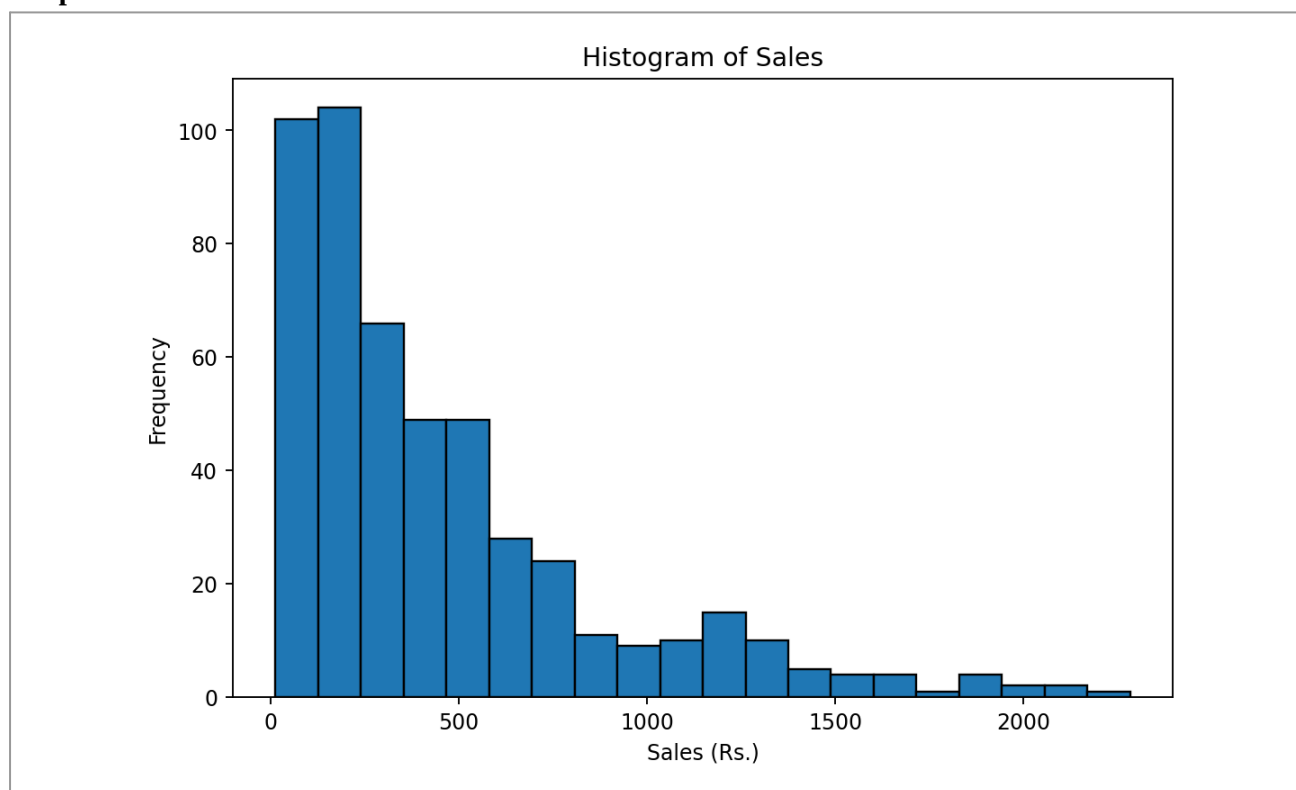
The histograms show the distribution and shape of Sales and customer Rating.

Code

```
# Histogram 1: Sales
plt.figure(figsize=(7,5))
plt.hist(df["Sales"], bins=20, edgecolor="black")
plt.title("Histogram of Sales")
plt.xlabel("Sales (Rs.)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Histogram 2: Rating
plt.figure(figsize=(7,5))
plt.hist(df["Rating"], bins=10, edgecolor="black", color="green")
plt.title("Histogram of Customer Rating")
plt.xlabel("Rating")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output



Sagar Kandari T010

Observation: Sales is right-skewed – most bills are small and a few bills are very large. Ratings are spread fairly evenly between 4 and 10.

6.4 Box Plot, Pie Chart and Scatter Plot

The box plot shows the median, spread and outliers of Sales; the pie chart shows the share of each payment method; the scatter plot shows how Sales changes with Quantity.

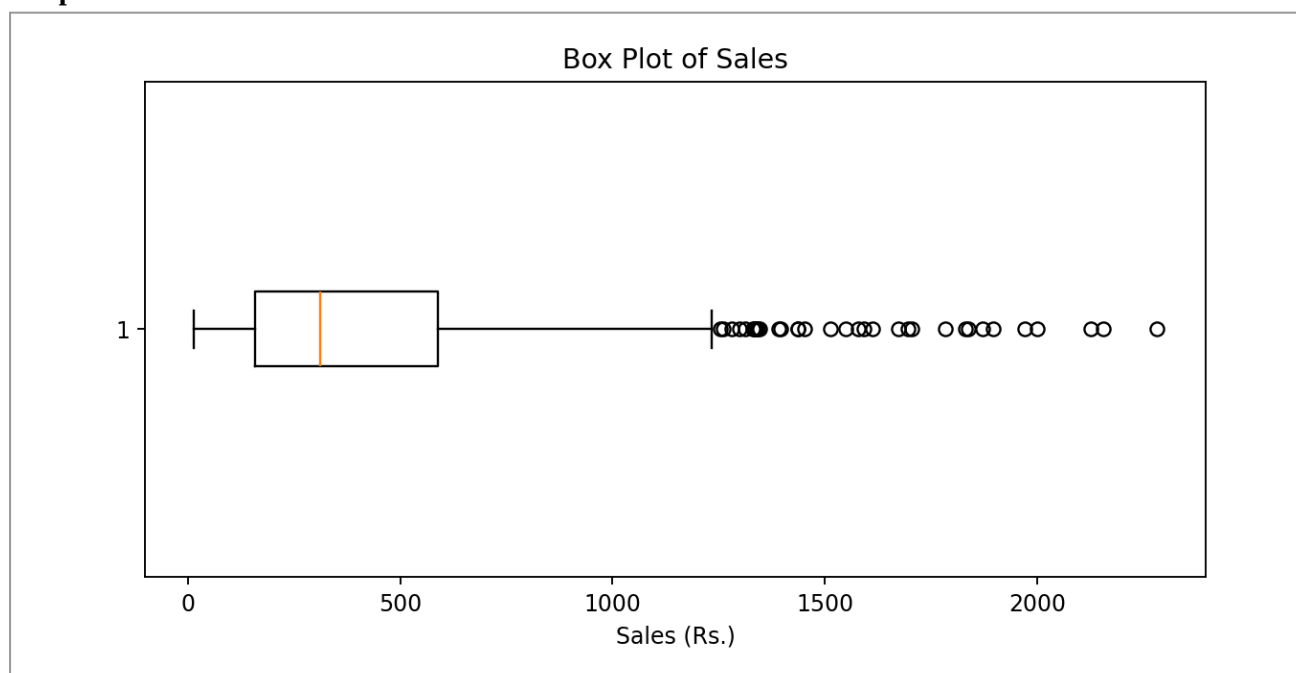
Code

```
# Box plot
plt.figure(figsize=(7,4))
plt.boxplot(df["Sales"], vert=False)
plt.title("Box Plot of Sales")
plt.xlabel("Sales (Rs.)")
plt.tight_layout()
plt.show()

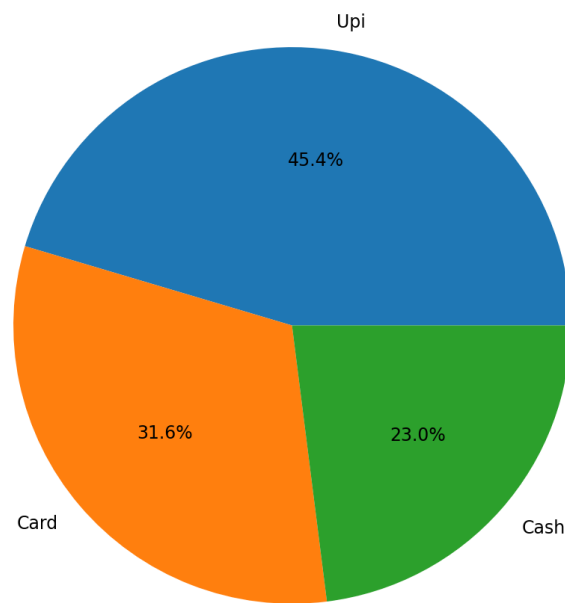
# Pie chart
plt.figure(figsize=(6,6))
df["Payment"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Payment Method Share")
plt.ylabel("")
plt.tight_layout()
plt.show()

# Scatter plot
plt.figure(figsize=(7,5))
plt.scatter(df["Quantity"], df["Sales"])
plt.title("Quantity vs Sales")
plt.xlabel("Quantity")
plt.ylabel("Sales (Rs.)")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

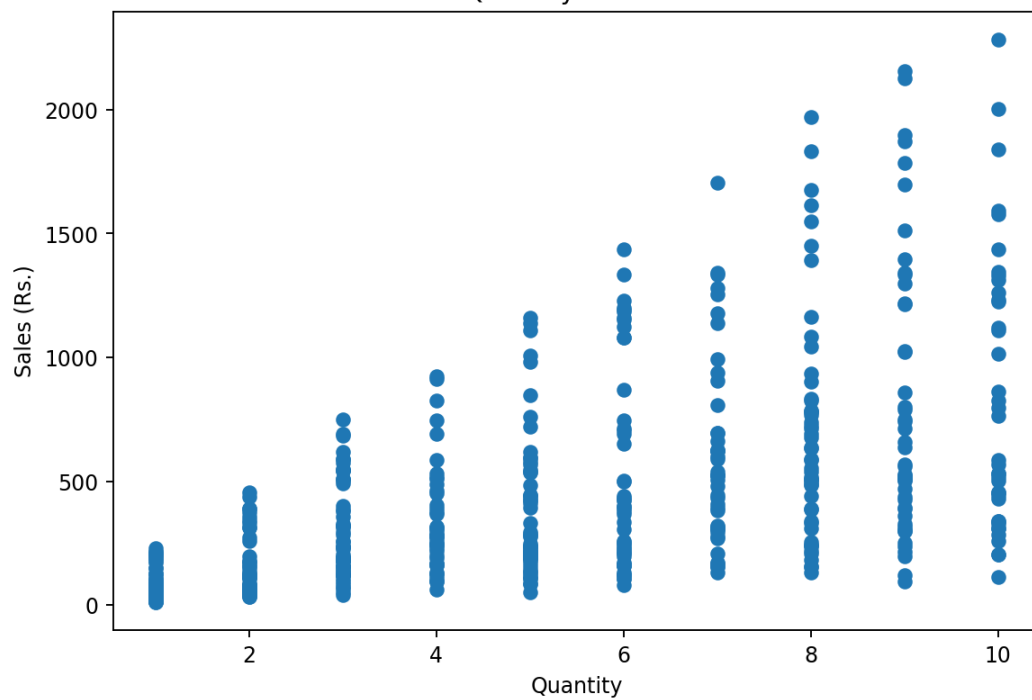
Output



Payment Method Share



Quantity vs Sales



Sagar Kandari T010

Observation: The box plot shows many high outliers in Sales. UPI is the most used payment method (about 45%), followed by Card and Cash. The scatter plot shows that sales rise as quantity increases.

7. Descriptive Statistics

7.1 Summary of Key Numeric Variables

Mean, median, mode, minimum, maximum, range, variance and standard deviation are calculated for Unit_Price, Quantity, Sales and Rating.

Code

```
cols = ["Unit_Price", "Quantity", "Sales", "Rating"]

desc = pd.DataFrame({
    "Mean": df[cols].mean(),
    "Median": df[cols].median(),
    "Mode": df[cols].mode().iloc[0],
    "Minimum": df[cols].min(),
    "Maximum": df[cols].max(),
    "Range": df[cols].max() - df[cols].min(),
    "Variance": df[cols].var(),
    "Std Dev": df[cols].std()
})
print(desc.round(2))
print("Sagar Kandari T010")
```

Output

	Mean	Median	Mode	Minimum	Maximum	Range	Variance	Std Dev
Unit_Price	85.76	66.16	26.44	10.57	249.34	238.77	3608.07	60.07
Quantity	5.31	5.00	3.00	1.00	10.00	9.00	7.99	2.83
Sales	455.42	311.06	45.80	13.05	2283.30	2270.25	183007.26	427.79
Rating	7.15	7.20	8.50	4.00	10.00	6.00	2.81	1.68
Sagar Kandari T010								

Interpretation: Mean sales (about Rs. 455) is higher than the median (about Rs. 311), so a few large bills pull the average up. The standard deviation is large, which means bill amounts vary a lot. Average quantity is about 5 units per bill.

8. Distribution Analysis

8.1 Distribution of Sales

A histogram and box plot of Sales are used to study the shape, central tendency, spread, outliers and skewness.

Code

```
plt.figure(figsize=(7,5))
plt.hist(df["Sales"], bins=20, edgecolor="black")
plt.title("Distribution of Sales")
plt.xlabel("Sales (Rs.)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

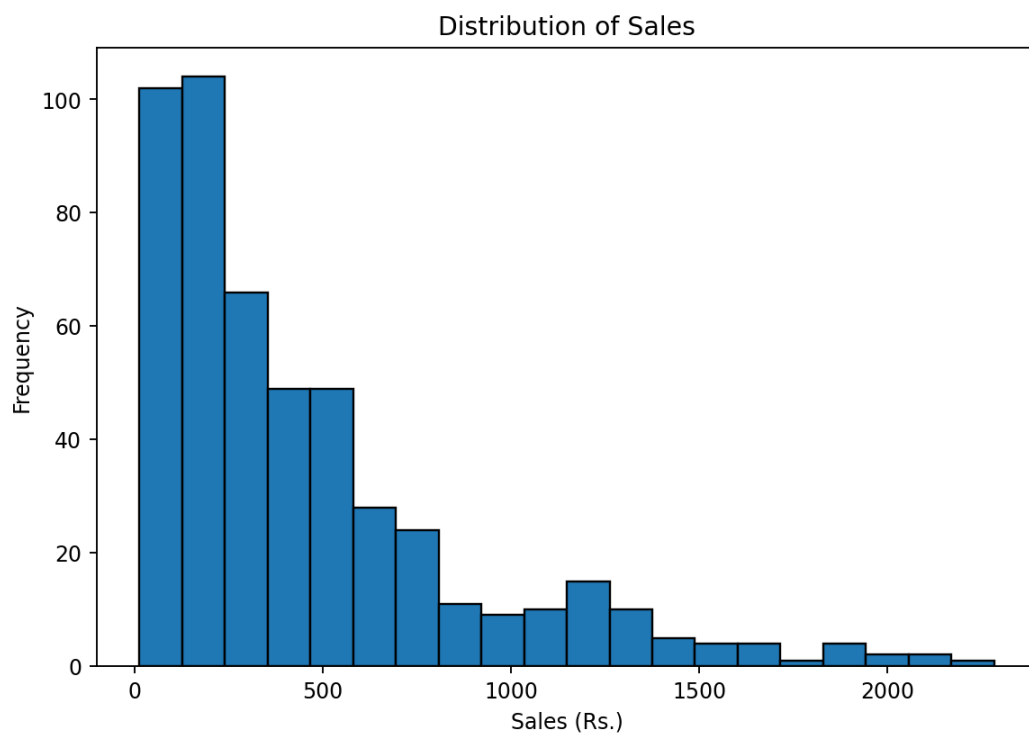
plt.figure(figsize=(7,4))
plt.boxplot(df["Sales"], vert=False)
plt.title("Box Plot of Sales")
plt.xlabel("Sales (Rs.)")
plt.tight_layout()
plt.show()

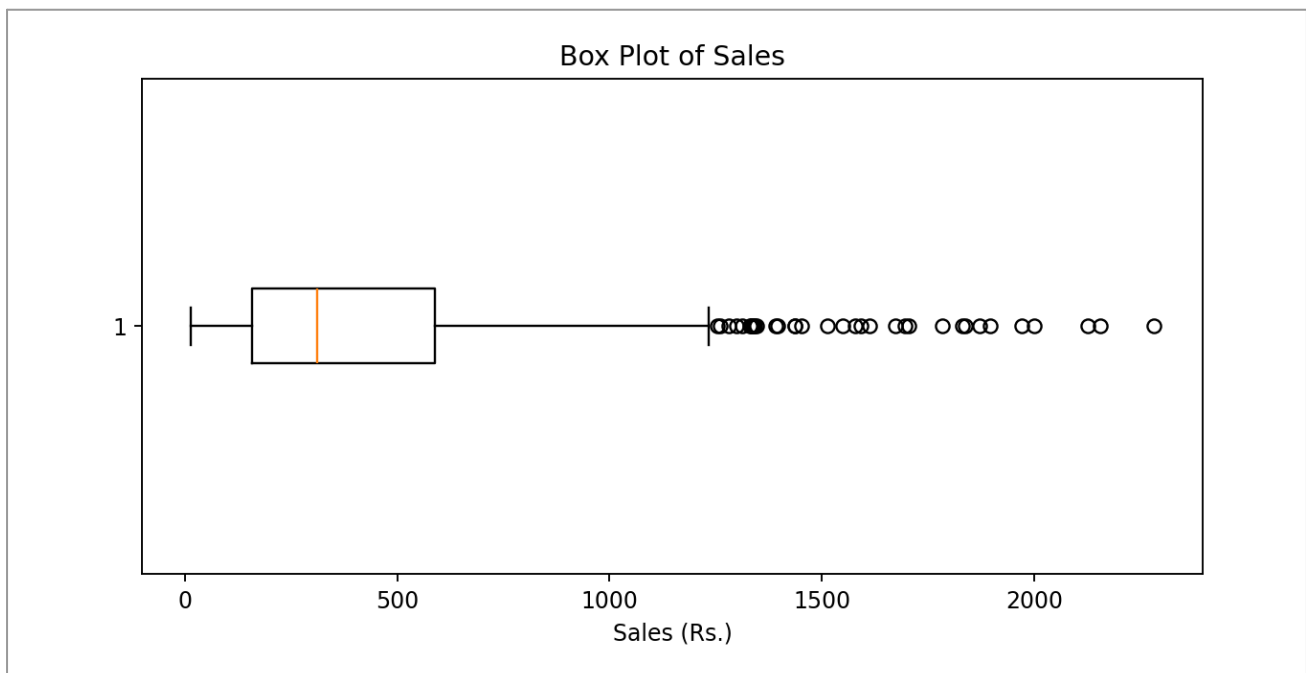
print("Mean:", round(df["Sales"].mean(), 2))
print("Median:", round(df["Sales"].median(), 2))
print("Standard Deviation:", round(df["Sales"].std(), 2))
```

```
print("Skewness:", round(df["Sales"].skew(), 2))
print("Outliers:", len(df[(df["Sales"] < lower) | (df["Sales"] > upper)]))
print("Sagar Kandari T010")
```

Output

```
Mean: 455.42
Median: 311.06
Standard Deviation: 427.79
Skewness: 1.64
Outliers: 35
Sagar Kandari T010
```





Interpretation: Sales is positively skewed (skewness about 1.64). Mean > median, the spread is wide and there are 35 outliers on the higher side.

9. Correlation Analysis

9.1 Correlation Matrix and Heatmap

A positive correlation means two variables increase together; a negative correlation means an inverse relationship.

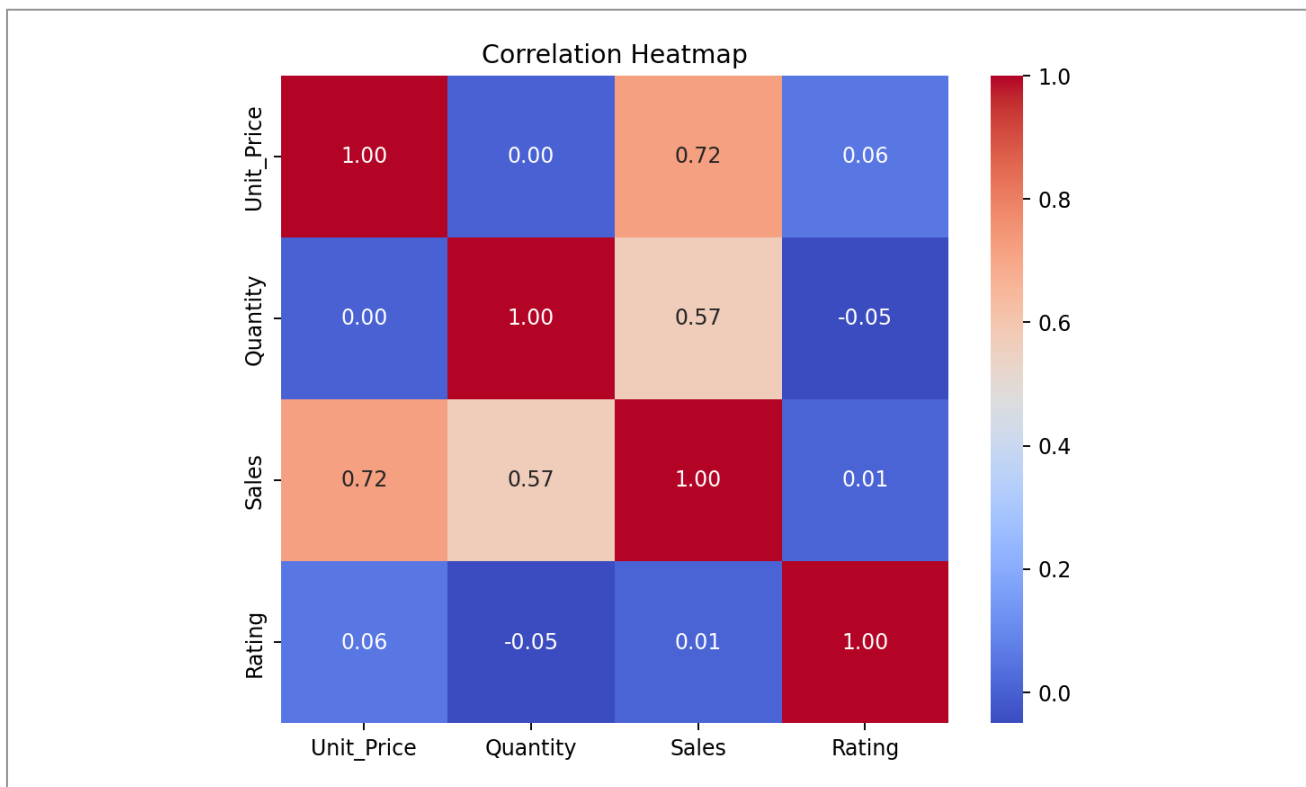
Code

```
corr = df[["Unit_Price", "Quantity", "Sales", "Rating"]].corr()
print("Correlation Matrix:")
print(corr.round(2))

plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

```
Correlation Matrix:
          Unit_Price  Quantity  Sales  Rating
Unit_Price      1.00      0.00   0.72   0.06
Quantity         0.00      1.00   0.57  -0.05
Sales            0.72      0.57   1.00   0.01
Rating           0.06     -0.05   0.01   1.00
Sagar Kandari T010
```



Interpretation: Sales has a strong positive relationship with Unit_Price (0.72) and a moderate positive relationship with Quantity (0.57). Rating has almost no relationship with any variable. Unit_Price and Quantity are not related to each other (0.00). No variable has a notable negative relationship.

10. Regression Analysis

A simple linear regression is built with **Quantity** as the independent variable and **Sales** as the dependent variable. **Sales = Intercept + Coefficient × Quantity**

10.1 Regression Model and Line

The model is fitted; its equation, coefficient, R^2 , MSE, RMSE and a prediction for 8 units are printed, and the regression line is plotted over the actual data.

Code

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = df[["Quantity"]]
y = df["Sales"]

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

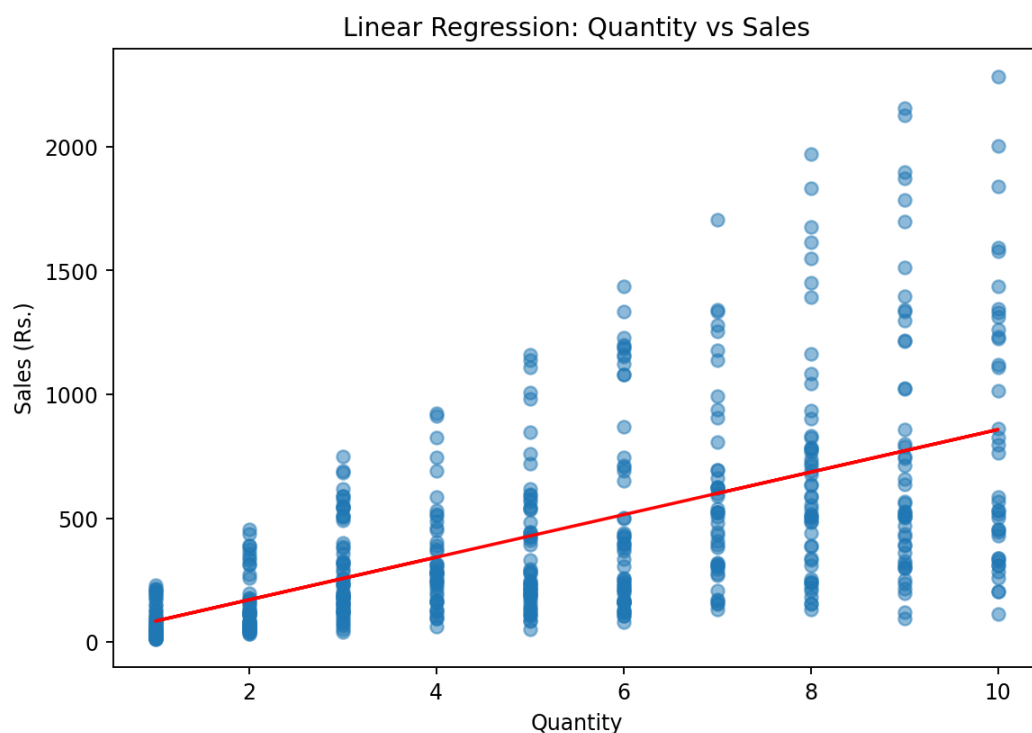
print("Regression Equation: Sales =", round(model.intercept_, 2), "+", round(model.coef_[0], 2), "x Quantity")
print("Regression Coefficient:", round(model.coef_[0], 2))
print("R2 Score:", round(r2_score(y, y_pred), 3))
mse = mean_squared_error(y, y_pred)
print("MSE:", round(mse, 2))
print("RMSE:", round(np.sqrt(mse), 2))

# Prediction for Quantity = 8
pred = model.predict(pd.DataFrame({"Quantity": [8]}))
print("Predicted Sales for Quantity 8:", round(pred[0], 2))
```

```
plt.figure(figsize=(7,5))
plt.scatter(X, y, alpha=0.5)
plt.plot(X, y_pred, color="red")
plt.title("Linear Regression: Quantity vs Sales")
plt.xlabel("Quantity")
plt.ylabel("Sales (Rs.)")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

```
Regression Equation: Sales = -0.42 + 85.88 x Quantity
Regression Coefficient: 85.88
R2 Score: 0.322
MSE: 123809.29
RMSE: 351.87
Predicted Sales for Quantity 8: 686.6
Sagar Kandari T010
```



Interpretation: For every extra unit purchased, sales increase by about Rs. 86. R^2 is about 0.32, so quantity explains about 32% of the variation in sales; the rest depends on price. The model predicts about Rs. 687 for a bill of 8 units.

11. Business Intelligence Dashboard

11.1 Dashboard Data

The cleaned data is saved as `dashboard_data.csv` so that it can be loaded into Tableau or Power BI.

Code

```
# Save cleaned data to import into Tableau / Power BI
df.to_csv("dashboard_data.csv", index=False)
print("dashboard_data.csv saved successfully")
print("Sagar Kandari T010")
```

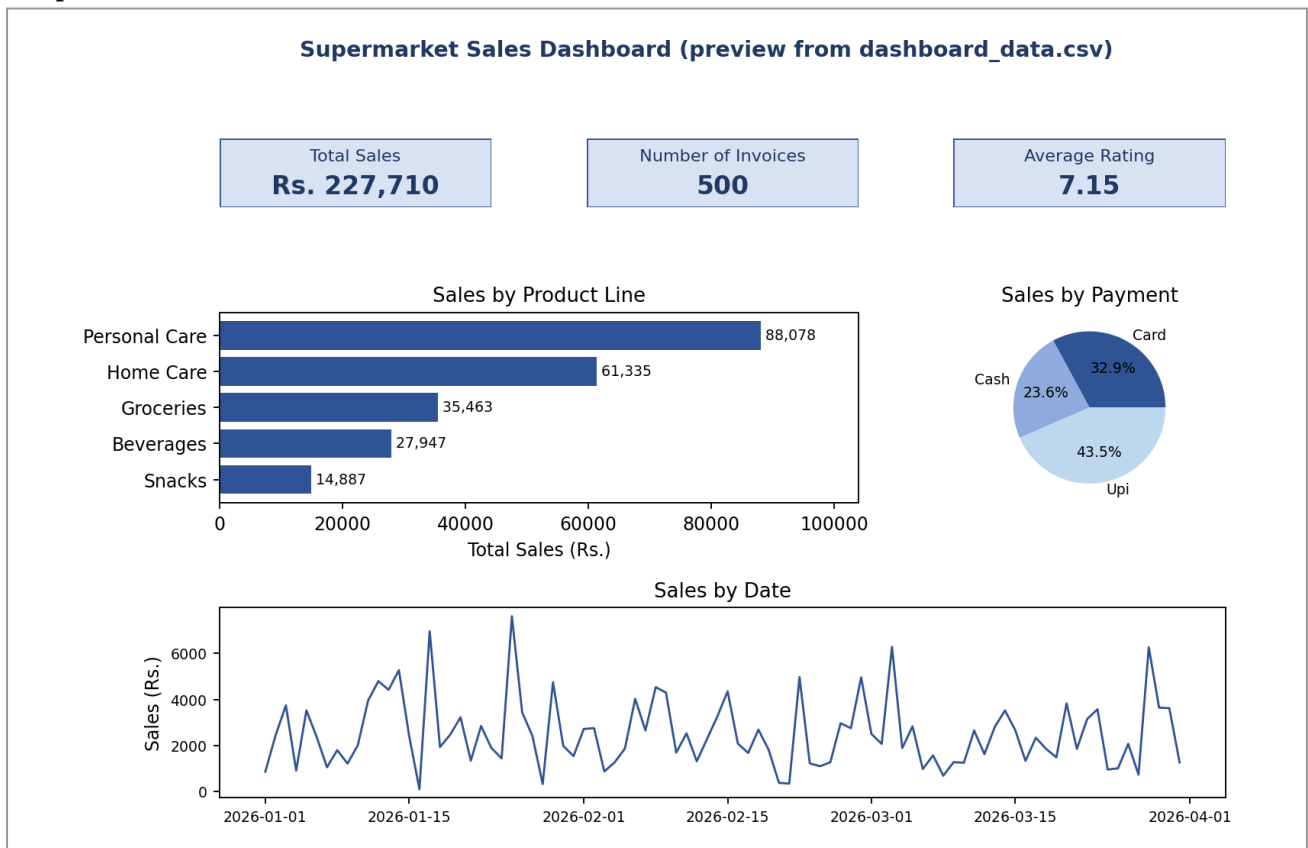
Output

```
dashboard_data.csv saved successfully
Sagar Kandari T010
```

11.2 Dashboard Design and Insights

In Power BI or Tableau, dashboard_data.csv is loaded and the following are created: **KPI cards** (Total Sales, Number of Invoices, Average Rating), a **bar chart** of Sales by Product_Line, a **pie chart** of Sales by Payment, a **line chart** of Sales by Date, and **filters** for Branch, Customer_Type and Product_Line, with titles and data labels on each chart.

Output



Note: The figure above is a Python preview of the dashboard layout built from dashboard_data.csv. No Tableau/Power BI screenshot was supplied with the project notebook, so none is claimed here.

Insights from the dashboard:

- Personal Care is the top product line by sales.
- Andheri and Borivali are the strongest branches.
- UPI is the most used payment method.
- January had the highest sales of the three months.
- Members spend more per bill than Normal customers.

12. Probability and Probability Distributions

12.1 Probability Experiments and Distributions

A dice-rolling experiment and dataset-based probabilities are calculated, and random values are generated from the Normal, Uniform and Exponential distributions.

Code

```
np.random.seed(1)

# Dice rolling experiment
rolls = np.random.randint(1, 7, 1000)
print("P(rolling a 6):", (rolls == 6).mean())
print("P(even number):", (rolls % 2 == 0).mean())

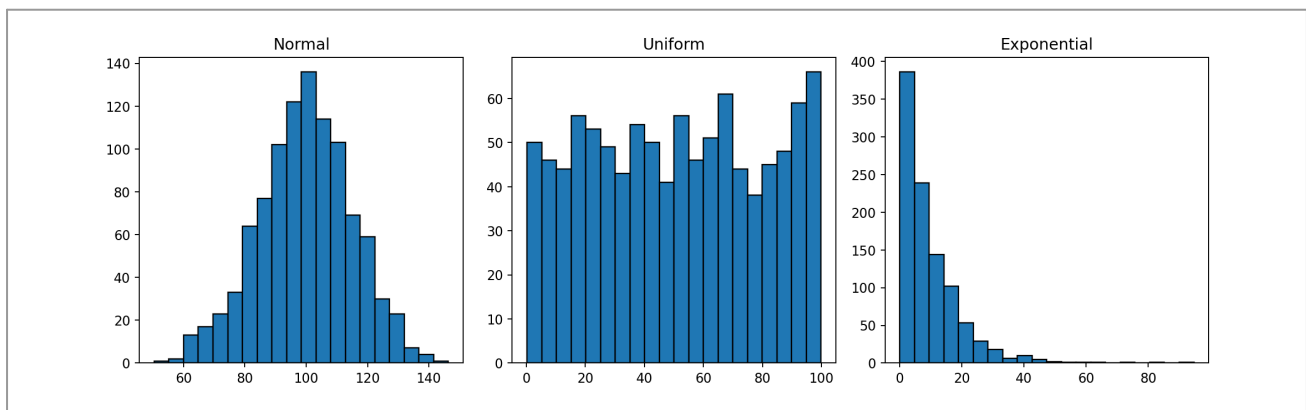
# Customer purchase probabilities from the dataset
print("P(customer is Member):", round((df["Customer_Type"] == "Member").mean(), 3))
print("P(Sales above 500):", round((df["Sales"] > 500).mean(), 3))
print("P(UPI payment):", round((df["Payment"] == "UPI").mean(), 3))

# Random values from three distributions
normal = np.random.normal(100, 15, 1000)
uniform = np.random.uniform(0, 100, 1000)
exponential = np.random.exponential(10, 1000)

plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.hist(normal, bins=20, edgecolor="black"); plt.title("Normal")
plt.subplot(1,3,2); plt.hist(uniform, bins=20, edgecolor="black"); plt.title("Uniform")
plt.subplot(1,3,3); plt.hist(exponential, bins=20, edgecolor="black"); plt.title("Exponential")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

```
P(rolling a 6): 0.17
P(even number): 0.496
P(customer is Member): 0.464
P(Sales above 500): 0.342
P(UPI payment): 0.454
Sagar Kandari T010
```



Interpretation: The dice probability is close to 1/6 and the even-number probability is close to 0.5. About 46% of customers are Members and 34% of bills are above Rs. 500. Normal data is bell shaped, Uniform data is flat, and Exponential data has many small values and a long tail to the right (like waiting times).

13. Decision Making Under Uncertainty

Expected Value = Probability $\times$ Payoff. The alternative with the highest expected value is preferred. Payoffs are in Rs. lakh for High (0.3), Medium (0.5) and Low (0.2) demand.

13.1 Decision Tree

Decision Tree

```
Decision
|-- Launch Premium Product -> High (0.3): +80 | Medium (0.5): +30 | Low (0.2): -40   EV = 31.0
|-- Launch Budget Product  -> High (0.3): +50 | Medium (0.5): +25 | Low (0.2): -10   EV = 25.5
|-- Do Not Launch          -> 0                                                    EV = 0
```

13.2 Expected Value of Each Alternative

Code

```
# Payoffs in Rs. lakh with probabilities (High, Medium, Low demand)
prob = [0.3, 0.5, 0.2]

payoff = {
    "Launch Premium Product": [80, 30, -40],
    "Launch Budget Product": [50, 25, -10],
    "Do Not Launch": [0, 0, 0]
}

expected = {}
for alt in payoff:
    ev = sum(p * v for p, v in zip(prob, payoff[alt]))
    expected[alt] = ev
    print(alt, "-> Expected Value =", round(ev, 2), "lakh")

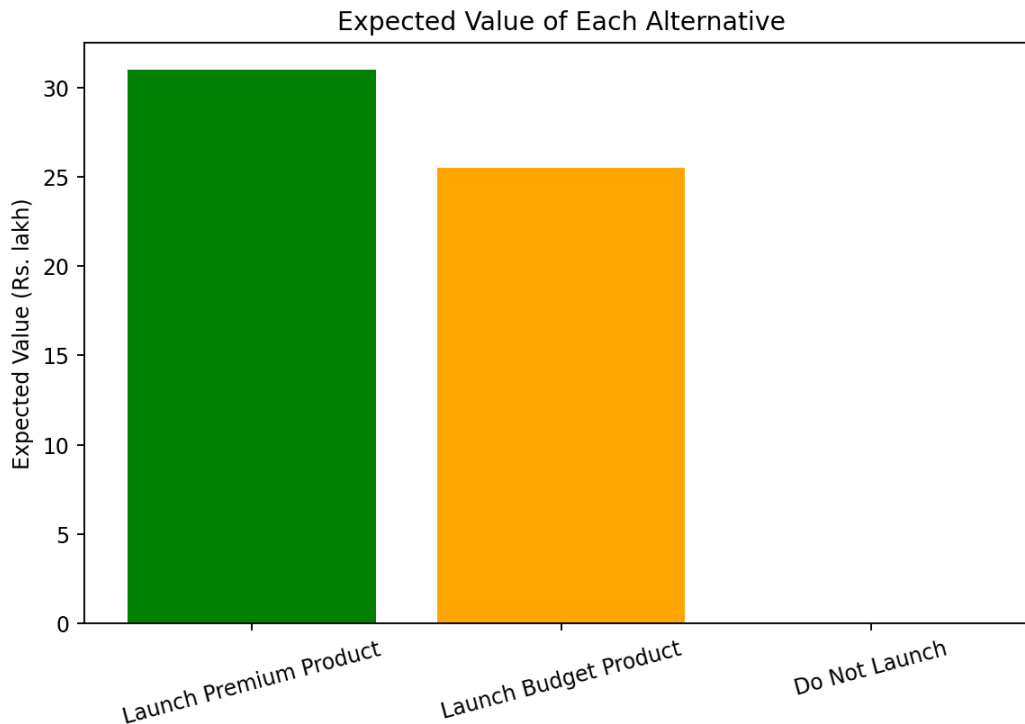
best = max(expected, key=expected.get)
print("\nBest Decision:", best)

plt.figure(figsize=(7,5))
plt.bar(expected.keys(), expected.values(), color=["green", "orange", "grey"])
plt.title("Expected Value of Each Alternative")
plt.ylabel("Expected Value (Rs. lakh)")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

```
Launch Premium Product -> Expected Value = 31.0 lakh  
Launch Budget Product -> Expected Value = 25.5 lakh  
Do Not Launch -> Expected Value = 0.0 lakh
```

```
Best Decision: Launch Premium Product  
Sagar Kandari T010
```



Interpretation: The Premium product has the highest expected value (Rs. 31 lakh), but it also has the largest possible loss. The Budget product is safer.

14. Sampling and Sampling Distribution

Population: all 500 invoices. **Sample:** 30 invoices. **Technique:** simple random sampling. **Repeated:** 100 times.

14.1 Sampling Distribution and Sample Sizes

Sample means are calculated for 100 random samples, a histogram of the sample means is drawn, and different sample sizes are compared.

Code

```
np.random.seed(42)  
sales = df["Sales"]  
  
print("Population Size:", len(sales))  
print("Population Mean:", round(sales.mean(), 2))  
  
# 100 samples of size 30  
sample_means = []  
for i in range(100):
```

```

sample = sales.sample(n=30, replace=False)
sample_means.append(sample.mean())

print("\nMean of Sample Means:", round(np.mean(sample_means), 2))
print("Standard Deviation of Sample Means:", round(np.std(sample_means, ddof=1), 2))
print("Standard Error:", round(sales.std() / np.sqrt(30), 2))

plt.figure(figsize=(7,5))
plt.hist(sample_means, bins=10, edgecolor="black")
plt.title("Sampling Distribution of Sample Means (n = 30)")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Different sample sizes
results = []
for n in [10, 20, 30, 40]:
    means = []
    for i in range(100):
        means.append(sales.sample(n=n, replace=False).mean())
    results.append({"Sample Size": n,
                    "Mean of Sample Means": np.mean(means),
                    "SD of Sample Means": np.std(means, ddof=1)})

print("\nComparison of Different Sample Sizes:")
print(pd.DataFrame(results).round(2))
print("Sagar Kandari T010")

```

Output

```

Population Size: 500
Population Mean: 455.42

```

```

Mean of Sample Means: 448.87
Standard Deviation of Sample Means: 84.75
Standard Error: 78.1

```

```

Comparison of Different Sample Sizes:

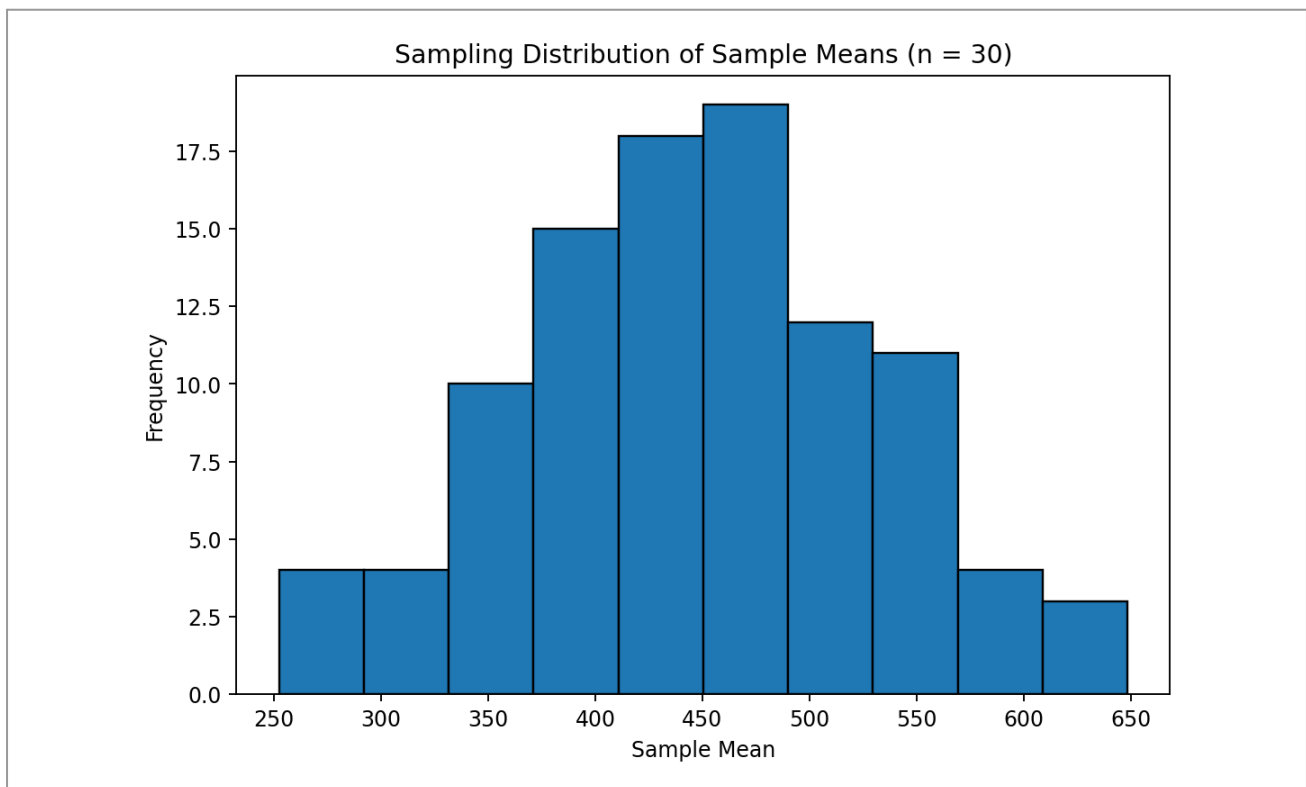
```

	Sample Size	Mean of Sample Means	SD of Sample Means
0	10	453.74	130.65
1	20	461.10	95.64
2	30	455.36	81.10
3	40	463.51	57.19

```

Sagar Kandari T010

```



Central Limit Theorem: When many random samples are taken, the sample means form a bell-shaped distribution centred on the population mean. As sample size increases, the spread (standard error) becomes smaller – as seen above, the SD falls from $n = 10$ to $n = 40$.

15. Confidence Interval Estimation

15.1 90%, 95% and 99% Confidence Intervals

t-intervals are calculated for the mean Sales per invoice from a random sample of 100 invoices.

Code

```
np.random.seed(42)
sample = df["Sales"].sample(n=100, random_state=42)

mean = sample.mean()
se = stats.sem(sample)

results = []
for level in [0.90, 0.95, 0.99]:
    low, high = stats.t.interval(level, len(sample) - 1, loc=mean, scale=se)
    results.append({"Confidence Level": str(int(level * 100)) + "%",
                    "Lower Limit": low, "Upper Limit": high})

print("Sample Mean:", round(mean, 2))
print(pd.DataFrame(results).round(2))
print("Sagar Kandari T010")
```

Output

```
Sample Mean: 434.98
Confidence Level  Lower Limit  Upper Limit
0                90%        370.45    499.51
1                95%        357.87    512.09
2                99%        332.91    537.05
Sagar Kandari T010
```

Meaning of 95% CI: We are 95% confident that the true average sales per invoice lies between the lower and upper limits shown. A higher confidence level gives a wider interval.

16. Bootstrapping (1,000 Resamples)

16.1 Bootstrap Median and Interval

1,000 resamples are drawn with replacement to estimate the uncertainty of the median Sales per invoice.

Code

```
np.random.seed(42)
sales = df["Sales"]

boot_medians = []
for i in range(1000):
    boot_sample = sales.sample(n=len(sales), replace=True)
    boot_medians.append(boot_sample.median())

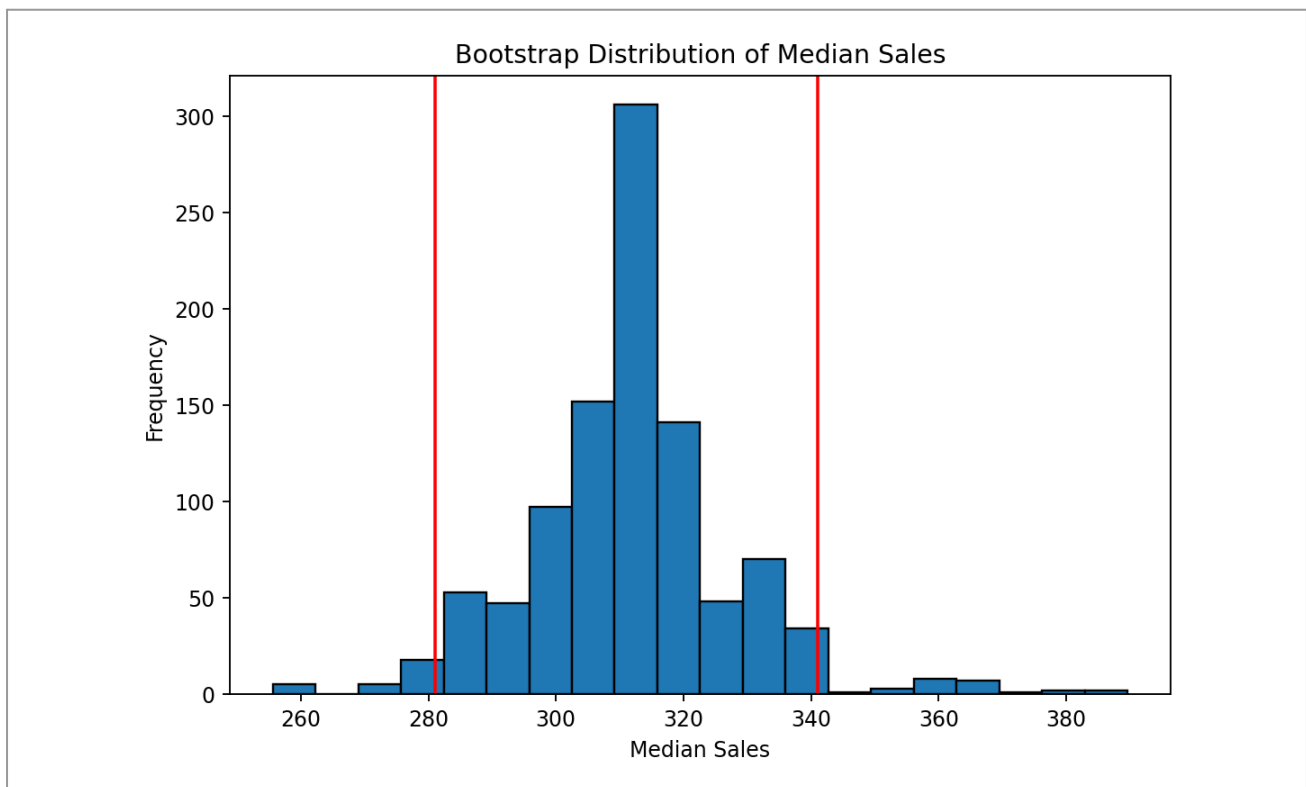
low = np.percentile(boot_medians, 2.5)
high = np.percentile(boot_medians, 97.5)

print("Original Median:", round(sales.median(), 2))
print("Mean of Bootstrap Medians:", round(np.mean(boot_medians), 2))
print("95% Bootstrap Confidence Interval:", round(low, 2), "to", round(high, 2))

plt.figure(figsize=(7,5))
plt.hist(boot_medians, bins=20, edgecolor="black")
plt.axvline(low, color="red")
plt.axvline(high, color="red")
plt.title("Bootstrap Distribution of Median Sales")
plt.xlabel("Median Sales")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

```
Original Median: 311.06
Mean of Bootstrap Medians: 311.95
95% Bootstrap Confidence Interval: 281.03 to 341.0
Sagar Kandari T010
```



Interpretation: After 1,000 resamples with replacement, the median sales per invoice is estimated at about Rs. 311, with a 95% interval of about Rs. 281 to Rs. 341.

17. Hypothesis Testing

Question: Is the average sales per invoice different from Rs. 300? H_0 : Mean sales per invoice = 300. H_1 : Mean sales per invoice $\neq$ 300. $\alpha = 0.05$ (one-sample t-test). If $p < 0.05$ reject H_0 ; otherwise fail to reject H_0 .

17.1 One-Sample t-Test

Code

```
# H0: Mean sales per invoice = 300
# H1: Mean sales per invoice is not equal to 300
alpha = 0.05

t_stat, p_value = stats.ttest_1samp(df["Sales"], 300)

print("Sample Mean:", round(df["Sales"].mean(), 2))
print("Test Statistic:", round(t_stat, 3))
print("p-value:", round(p_value, 4))
print("Significance Level:", alpha)

if p_value < alpha:
    print("Decision: Reject H0")
else:
    print("Decision: Fail to reject H0")
print("Sagar Kandari T010")
```

Output

```
Sample Mean: 455.42
Test Statistic: 8.124
p-value: 0.0
Significance Level: 0.05
Decision: Reject H0
Sagar Kandari T010
```

Business interpretation: The p-value is far below 0.05, so average sales per invoice is significantly different from Rs. 300 – it is higher (about Rs. 455).

18. A/B Testing

Version A (“Buy Now”) and Version B (“Get Started”) are compared with a two-proportion z-test, using 1,000 visitors for each version. H_0 : both buttons have the same conversion rate; H_1 : the conversion rates are different.

18.1 Conversion Rates and Two-Proportion z-Test

Code

```
# Version A: "Buy Now"    Version B: "Get Started"
visitors_a, visitors_b = 1000, 1000
conv_a, conv_b = 120, 150

rate_a = conv_a / visitors_a
rate_b = conv_b / visitors_b
print("Conversion Rate A:", rate_a)
print("Conversion Rate B:", rate_b)

# Two proportion z-test
p_pool = (conv_a + conv_b) / (visitors_a + visitors_b)
se = np.sqrt(p_pool * (1 - p_pool) * (1/visitors_a + 1/visitors_b))
z = (rate_b - rate_a) / se
p_value = 2 * (1 - stats.norm.cdf(abs(z)))

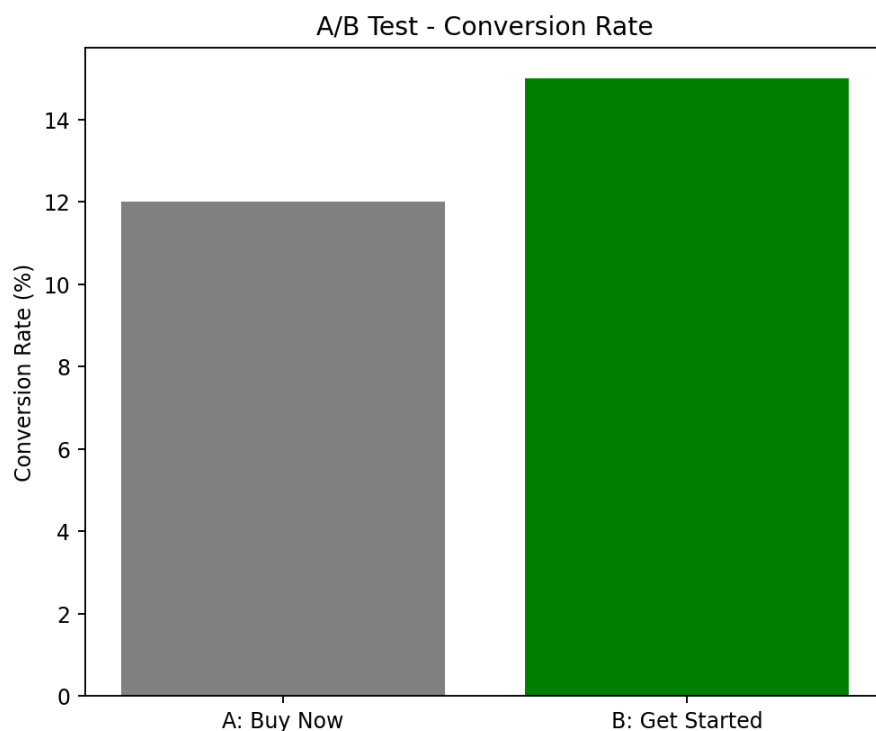
print("z-statistic:", round(z, 3))
print("p-value:", round(p_value, 4))

if p_value < 0.05:
    print("Decision: Reject H0 - the two versions perform differently")
else:
    print("Decision: Fail to reject H0 - no significant difference")

plt.figure(figsize=(6,5))
plt.bar(["A: Buy Now", "B: Get Started"], [rate_a * 100, rate_b * 100], color=["grey", "green"])
plt.title("A/B Test - Conversion Rate")
plt.ylabel("Conversion Rate (%)")
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

```
Conversion Rate A: 0.12
Conversion Rate B: 0.15
z-statistic: 1.963
p-value: 0.0496
Decision: Reject H0 - the two versions perform differently
Sagar Kandari T010
```



Business interpretation: Version B (“Get Started”) converts at 15% versus 12% for A. The result is only just significant (p is close to 0.05), so B can be adopted but should be tested a little longer.

19. Time-Series Analysis

19.1 Daily Sales, Trend and Weekly Pattern

The Date column is converted to datetime and set as the index, daily sales are totalled (missing dates filled with zero), the trend is shown with a 7-day moving average, and average sales by weekday are calculated.

Code

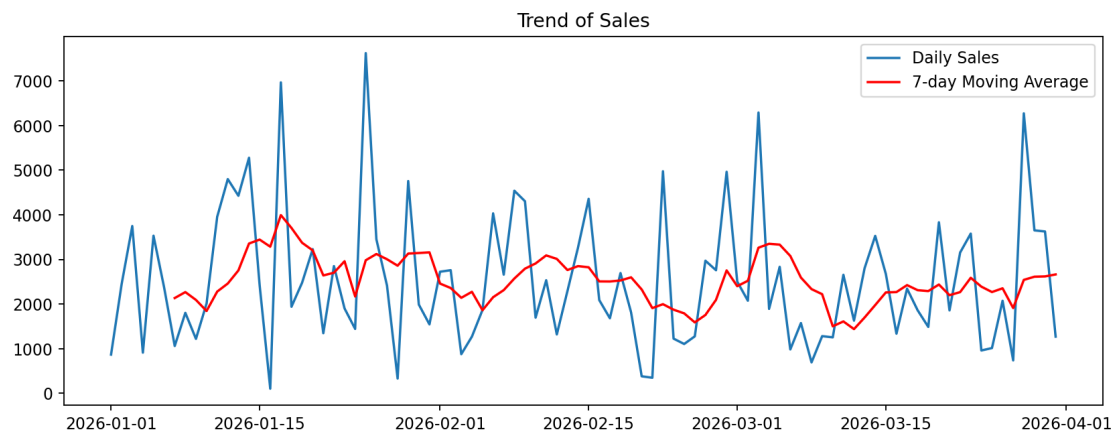
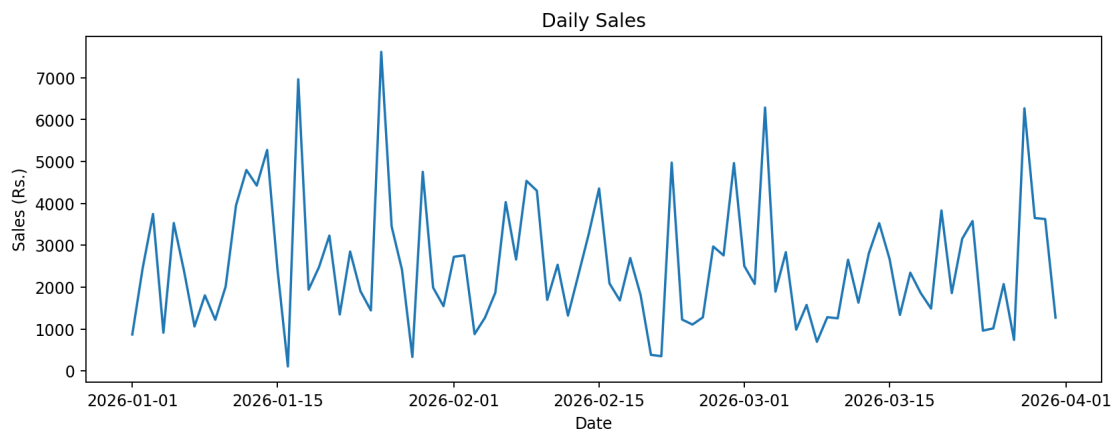
```
# Convert date, set as index and total daily sales
df["Date"] = pd.to_datetime(df["Date"])
daily = df.groupby("Date")["Sales"].sum()
daily = daily.asfreq("D").fillna(0)

plt.figure(figsize=(10,4))
plt.plot(daily)
plt.title("Daily Sales")
plt.xlabel("Date")
plt.ylabel("Sales (Rs.)")
plt.tight_layout()
plt.show()
```

```
# Trend using 7-day moving average
moving_avg = daily.rolling(7).mean()
plt.figure(figsize=(10,4))
plt.plot(daily, label="Daily Sales")
plt.plot(moving_avg, color="red", label="7-day Moving Average")
plt.title("Trend of Sales")
plt.legend()
plt.tight_layout()
plt.show()

# Seasonality: average sales by day of week
weekday = daily.groupby(daily.index.day_name()).mean()
print(weekday.round(2))
print("Sagar Kandari T010")
```

Output



```
Date
Friday      1960.46
Monday      2809.88
Saturday     3090.61
Sunday       3360.68
Thursday     2207.11
Tuesday      2301.72
Wednesday    1934.49
Name: Sales, dtype: float64
Sagar Kandari T010
```

Observation: Daily sales move up and down, with the highest average sales on Sunday and Saturday and the lowest on Wednesday, suggesting a weekly (weekend) pattern. January sales were the highest month.

19.2 Exponential Smoothing and ARIMA Forecasts

The last 14 days are kept for testing. Simple exponential smoothing and an ARIMA(1,1,1) model are fitted on the training data and used to forecast the next 14 days.

Code

```
# pip install statsmodels (run once if not installed)
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
from statsmodels.tsa.arima.model import ARIMA

# Keep last 14 days for testing
train = daily[:-14]
test = daily[-14:]

# Exponential smoothing forecast
es_model = SimpleExpSmoothing(train).fit()
es_forecast = es_model.forecast(14)

# ARIMA forecast
arima_model = ARIMA(train, order=(1, 1, 1)).fit()
arima_forecast = arima_model.forecast(14)

print("Exponential Smoothing Forecast:")
print(es_forecast.round(2).head())
print("\nARIMA Forecast:")
print(arima_forecast.round(2).head())

plt.figure(figsize=(10,4))
plt.plot(test, label="Actual")
plt.plot(es_forecast, label="Exponential Smoothing")
plt.plot(arima_forecast, label="ARIMA")
plt.title("Forecast Comparison")
plt.legend()
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

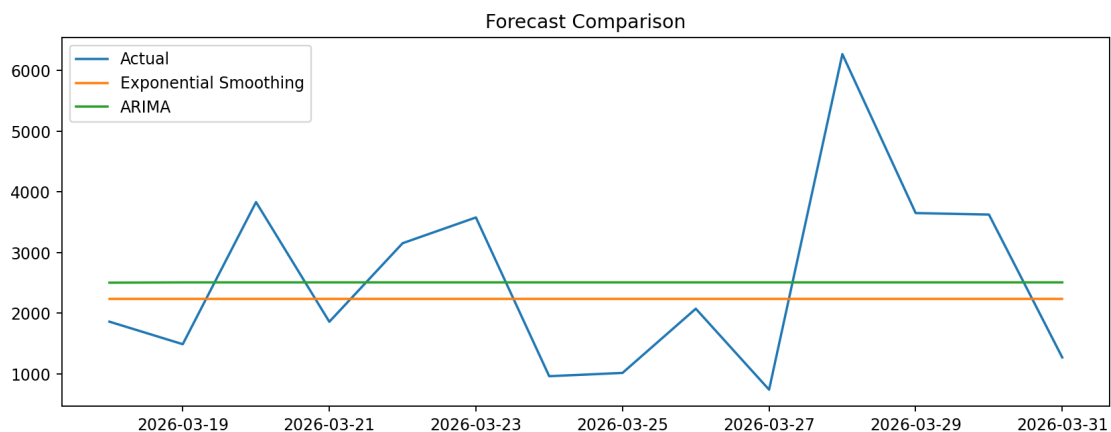
Output

Exponential Smoothing Forecast:

```
2026-03-18    2240.43
2026-03-19    2240.43
2026-03-20    2240.43
2026-03-21    2240.43
2026-03-22    2240.43
Freq: D, dtype: float64
```

ARIMA Forecast:

```
2026-03-18    2502.52
2026-03-19    2507.36
2026-03-20    2507.51
2026-03-21    2507.51
2026-03-22    2507.51
Freq: D, dtype: float64
Sagar Kandari T010
```



Observation: Exponential smoothing gives an almost flat forecast near the recent average. ARIMA also gives a smooth forecast. Both are compared with the actual values in the chart above.

20. Forecast Accuracy

20.1 MAE, MSE and RMSE

The two forecasts are evaluated on the 14 test days using MAE, MSE and RMSE.

Code

```
from sklearn.metrics import mean_absolute_error

rows = []
for name, forecast in [("Exponential Smoothing", es_forecast), ("ARIMA", arima_forecast)]:
    mae = mean_absolute_error(test, forecast)
    mse = mean_squared_error(test, forecast)
    rows.append({"Model": name, "MAE": mae, "MSE": mse, "RMSE": np.sqrt(mse)})

print(pd.DataFrame(rows).round(2))
print("Sagar Kandari T010")
```

Output

	Model	MAE	MSE	RMSE
0	Exponential Smoothing	1237.36	2322266.56	1523.90
1	ARIMA	1275.17	2240440.20	1496.81

Sagar Kandari T010

Interpretation: MAE is the average error in rupees, and RMSE gives more weight to large errors. Exponential Smoothing has the lower MAE, while ARIMA has the lower RMSE, so the preferred model depends on whether average error or sensitivity to large errors matters more.

21. Optimization Modeling

Decision variables: A, B = units of Product A and Product B. **Objective:** Maximize Profit = $40A + 30B$. **Constraints:** $2A + B \leq 100$ (labour), $A + 2B \leq 80$ (machine). **Non-negativity:** $A, B \geq 0$.

21.1 Linear Programming Solution

Code

```
from scipy.optimize import linprog

# Maximize Profit = 40A + 30B
# Labour: 2A + 1B <= 100
# Machine: 1A + 2B <= 80
# A, B >= 0
c = [-40, -30] # negative because linprog minimizes
A_ub = [[2, 1], [1, 2]]
b_ub = [100, 80]

result = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=(0, None))

A, B = result.x
print("Optimal units of Product A:", round(A, 2))
print("Optimal units of Product B:", round(B, 2))
print("Maximum Profit: Rs.", round(-result.fun, 2))
print("Labour used:", round(2*A + B, 2), "out of 100")
print("Machine used:", round(A + 2*B, 2), "out of 80")
print("Sagar Kandari T010")
```

Output

```
Optimal units of Product A: 40.0
Optimal units of Product B: 20.0
Maximum Profit: Rs. 2200.0
Labour used: 100.0 out of 100
Machine used: 80.0 out of 80
Sagar Kandari T010
```

Business meaning: Produce 40 units of A and 20 units of B for a maximum profit of Rs. 2,200. Both labour and machine hours are fully used, so adding either resource would increase profit.

22. Simulation Modeling

An inventory simulation is run for 365 days with random daily demand (average 40 units). Three scenarios are compared: a stock of 35, 45 and 55 units per day.

22.1 Inventory Simulation

Code

```
np.random.seed(5)

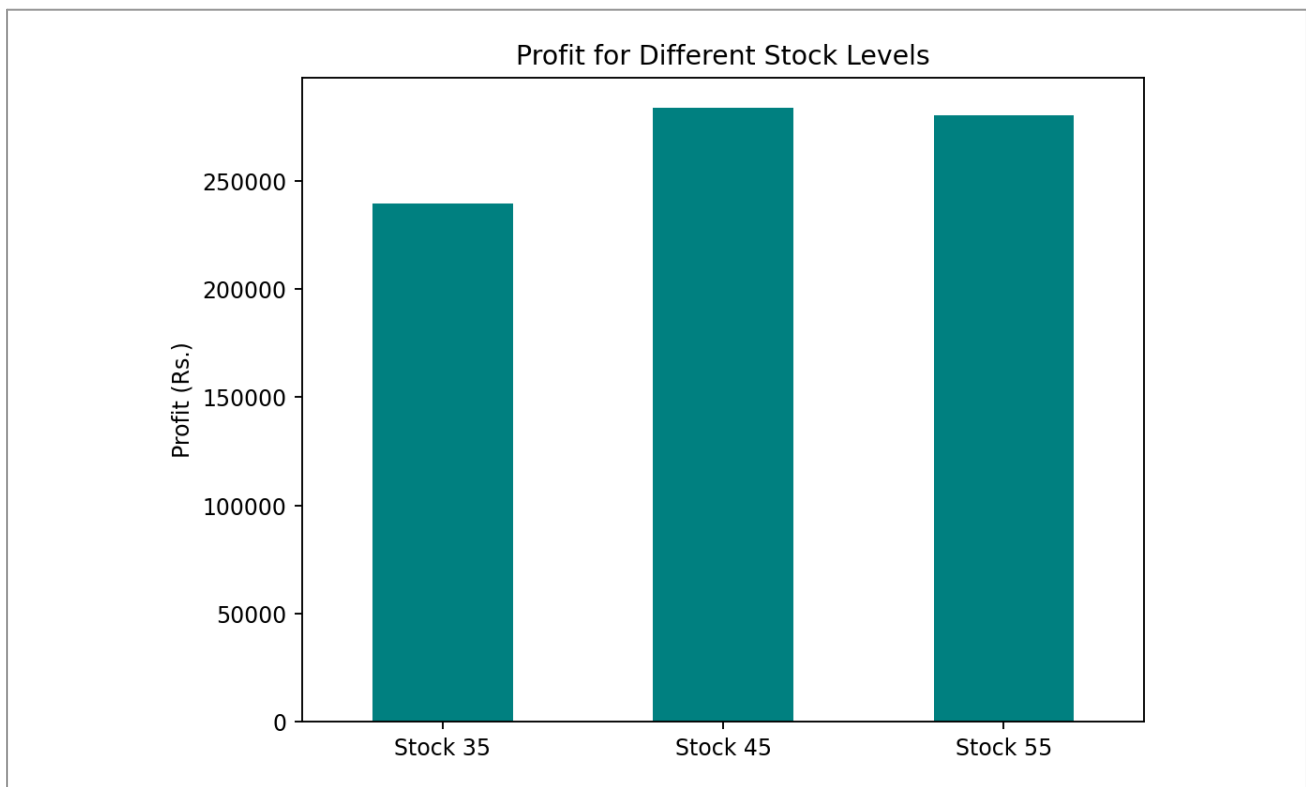
def simulate(stock, days=365):
    demand = np.random.poisson(40, days)      # daily demand
    sold = np.minimum(demand, stock)
    shortage = np.maximum(demand - stock, 0)
    leftover = stock - sold
    profit = sold * 20 - shortage * 5 - leftover * 2
    return {"Total Sales (units)": sold.sum(),
            "Shortage (units)": shortage.sum(),
            "Average Inventory": leftover.mean(),
            "Service Level %": 100 * sold.sum() / demand.sum(),
            "Profit (Rs.)": profit.sum()}

scenarios = {"Stock 35": 35, "Stock 45": 45, "Stock 55": 55}
result = pd.DataFrame({name: simulate(s) for name, s in scenarios.items()}).T
print(result.round(1))

result["Profit (Rs.)"].plot(kind="bar", color="teal")
plt.title("Profit for Different Stock Levels")
plt.ylabel("Profit (Rs.)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

	Total Sales (units)	Shortage (units)	Average Inventory	Service Level %	Profit (Rs.)
Stock 35	12500.0	2017.0	0.8	86.1	239365.0
Stock 45	14439.0	273.0	5.4	98.1	283443.0
Stock 55	14559.0	11.0	15.1	99.9	280093.0
Sagar Kandari T010					



Interpretation: Stocking 35 units causes many shortages (service level about 86%). Stocking 45 units gives the highest profit with about 98% service level. Stocking 55 units adds very little service but extra leftover stock, so profit drops.

23. ANOVA

ANOVA tests whether mean Sales differs across payment methods (Cash, Card and UPI). H_0 : all group means are equal; H_1 : at least one group mean differs. $\alpha = 0.05$.

23.1 One-Way ANOVA

Code

```
# H0: Mean sales of Cash, Card and UPI payments are equal
# H1: At least one mean is different
cash = df[df["Payment"] == "Cash"]["Sales"]
card = df[df["Payment"] == "Card"]["Sales"]
upi = df[df["Payment"] == "UPI"]["Sales"]

f_stat, p_value = stats.f_oneway(cash, card, upi)

print("Mean Sales - Cash:", round(cash.mean(), 2), "Card:", round(card.mean(), 2), "UPI:", round(
    upi.mean(), 2))
print("F-statistic:", round(f_stat, 3))
print("p-value:", round(p_value, 4))
print("Significance Level: 0.05")

if p_value < 0.05:
    print("Decision: Reject H0")
else:
    print("Decision: Fail to reject H0")
print("Sagar Kandari T010")
```

Output

```
Mean Sales - Cash: 468.19 Card: 474.12 UPI: 435.93
F-statistic: 0.437
p-value: 0.6463
Significance Level: 0.05
Decision: Fail to reject H0
Sagar Kandari T010
```

Business interpretation: The p-value is above 0.05, so there is no significant difference in spending between payment methods. Promotions need not be different for each payment type.

24. Experimental Design

An experiment is designed to study the effect of advertising and discounts on sales.

- **Factor 1:** Advertising Type – Social Media, Email.
- **Factor 2:** Discount – 5%, 15%, 25%.
- **Response variable:** Average sales per store.
- **Experimental units:** 60 store-weeks (10 for each combination).
- **Conduct:** Each store-week is randomly assigned one advertising type and one discount level; sales are recorded at the end of the week.

24.1 Experiment Table

Code

```
# Experiment table: every combination of the two factors
factor1 = ["Social Media", "Email"]
factor2 = ["5% Discount", "15% Discount", "25% Discount"]

design = pd.DataFrame([(a, d) for a in factor1 for d in factor2],
                      columns=["Advertising Type", "Discount"])
design["Stores per combination"] = 10
print(design)
print("Total experimental units:", design["Stores per combination"].sum())
print("Sagar Kandari T010")
```

Output

```
Advertising Type  Discount  Stores per combination
0      Social Media    5% Discount                10
1      Social Media   15% Discount                10
2      Social Media   25% Discount                10
3           Email     5% Discount                10
4           Email    15% Discount                10
5           Email    25% Discount                10
Total experimental units: 60
Sagar Kandari T010
```

25. Factorial Experiment (2×3)

Factors: **Advertising Type** (Social, Email) and **Discount** (5%, 15%, 25%). Response: **Sales**. Each combination has 10 replications.

25.1 Two-Way ANOVA and Interaction Plot

Code

```
# pip install statsmodels (run once if not installed)
import statsmodels.api as sm
from statsmodels.formula.api import ols

np.random.seed(7)
rows = []
for ad, ad_effect in [("Social", 5), ("Email", 0)]:
    for disc, disc_effect in [(5, 0), (15, 10), (25, 15)]:
        for i in range(10):
            sales_value = 100 + ad_effect + disc_effect + np.random.normal(0, 5)
            rows.append([ad, disc, sales_value])

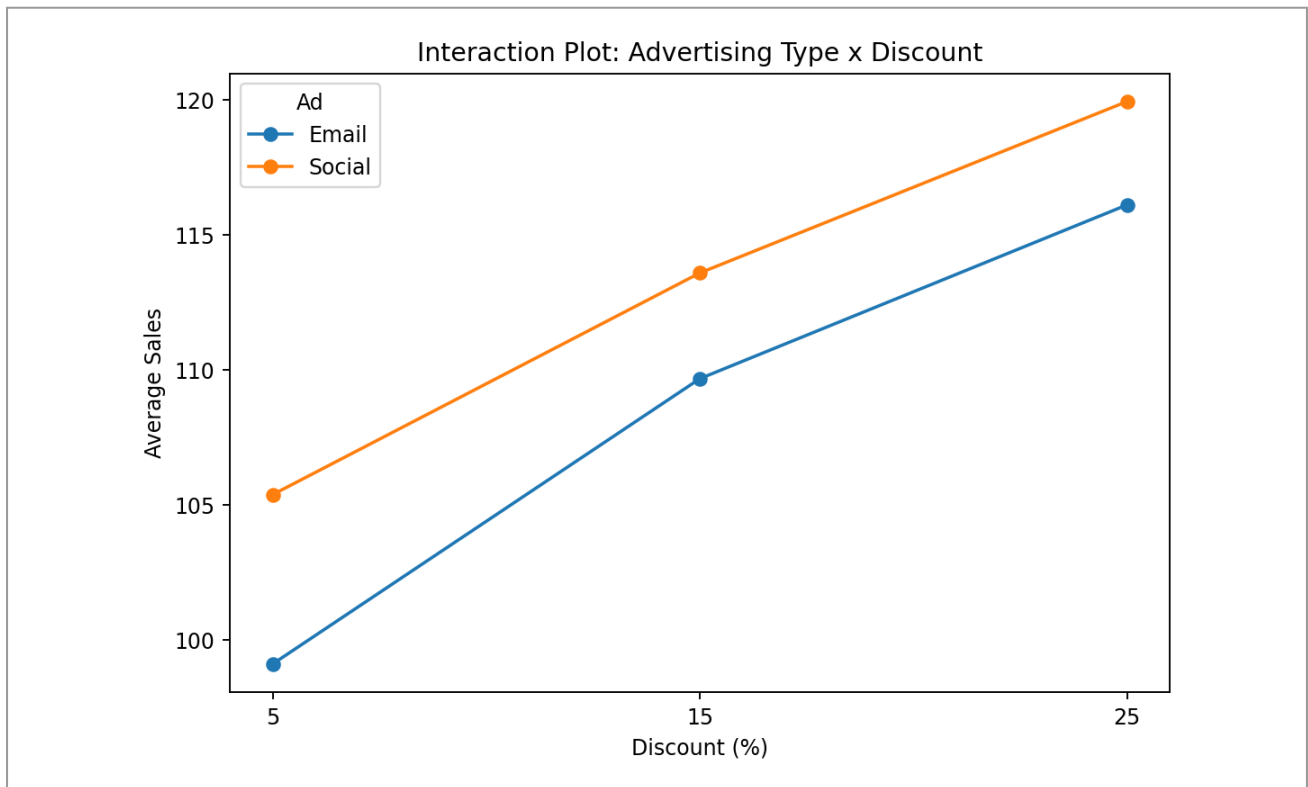
exp = pd.DataFrame(rows, columns=["Ad", "Discount", "Sales"])

model = ols("Sales ~ C(Ad) + C(Discount) + C(Ad):C(Discount)", data=exp).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
print(anova_table.round(4))

# Interaction plot
means = exp.groupby(["Discount", "Ad"])["Sales"].mean().unstack()
means.plot(marker="o", figsize=(7,5))
plt.title("Interaction Plot: Advertising Type x Discount")
plt.xlabel("Discount (%)")
plt.ylabel("Average Sales")
plt.xticks([5, 15, 25])
plt.tight_layout()
plt.show()
print("Sagar Kandari T010")
```

Output

	sum_sq	df	F	PR(>F)
C(Ad)	328.0578	1.0	10.8556	0.0017
C(Discount)	2525.8966	2.0	41.7916	0.0000
C(Ad):C(Discount)	19.2816	2.0	0.3190	0.7282
Residual	1631.8879	54.0	NaN	NaN
Sagar Kandari T010				



Interpretation: Advertising Type ($F = 10.86$, $p = 0.0017$) and Discount ($F = 41.79$, $p < 0.0001$) both significantly affect sales in the simulated experiment. The interaction ($F = 0.32$, $p = 0.7282$) is not significant, so the effect of discount does not materially depend on advertising type; the lines in the interaction plot are nearly parallel.

26. Business Insights and Findings

- **Personal Care** and **Home Care** give the highest sales (Rs. 88,078 and Rs. 61,335), because their average bill (Rs. 786 and Rs. 660) is much higher than Snacks (Rs. 164).
- **Thane** branch sales (Rs. 66,152) are about 18% lower than Andheri (Rs. 81,198) and Borivali (Rs. 80,360).
- **Member** customers spend more per bill (Rs. 480) than Normal customers (Rs. 434).
- **UPI** is the most used payment method (45.4%), ahead of Card (31.6%) and Cash (23.0%).
- Sales is right-skewed (skewness 1.64); mean Rs. 455 vs median Rs. 311, so a few large bills drive revenue.
- **Unit price** has a strong relationship with sales ($r = 0.72$) and **quantity** a moderate one ($r = 0.57$). Regression shows each extra unit adds about Rs. 86 ($R^2 = 0.322$).
- Average sales per invoice is significantly different from Rs. 300 ($t = 8.124$, $p < 0.05$).
- Payment method does **not** significantly change spending (ANOVA $p = 0.65$).
- Inventory simulation shows a stock level of 45 units/day gives the highest profit with about 98% service level.
- Button B (“Get Started”) converts better (15% vs 12%), at the edge of significance ($p = 0.0496$).
- Weekend sales are stronger than mid-week sales, especially Sunday (Rs. 3,361) and Saturday (Rs. 3,091) against Wednesday (Rs. 1,934).
- Both advertising type and discount level significantly affect sales in the factorial experiment, with no significant interaction.

27. Business Recommendations

- Promote Personal Care and Home Care products, since they give the highest sales per bill.
- Run local promotions in the Thane branch to bring it up to the level of the other branches.
- Encourage more customers to become Members, as Members spend more per bill.
- Keep UPI as the main payment option and offer UPI cashback offers.
- Keep around 45 units/day stock for fast-moving items to avoid shortages without overstocking.
- Use sales forecasts for weekly stock planning, with extra stock for weekends.
- Use the “Get Started” button on the website, and keep testing it.

28. Conclusion

This project studied the sales data of a supermarket. The data was cleaned and explored, and descriptive statistics, correlation, regression, probability, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series forecasting, optimization, simulation, ANOVA and factorial experiments were applied. The main findings are that Personal Care and Home Care drive sales, Thane lags behind, Members spend more, and a stock level of about 45 units gives the best profit. The supermarket can use these findings for stock planning, promotions and forecasting. Through this project I learned how different analytics techniques work together to support business decisions.

29. Appendix

The complete Python workflow – dataset creation/loading, preprocessing, EDA charts, statistical calculations, regression, probability distributions, decision-tree logic, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series forecasting, optimization, simulation, ANOVA and the factorial experiment – is contained in the project notebook **Business_Analytics_Project_Sagar_T010.ipynb**, which also generates `dashboard_data.csv` for Tableau/Power BI. All code and outputs are shown in the sections above.

The notebook does not contain an embedded Tableau/Power BI screenshot or a survey questionnaire, so neither is included.